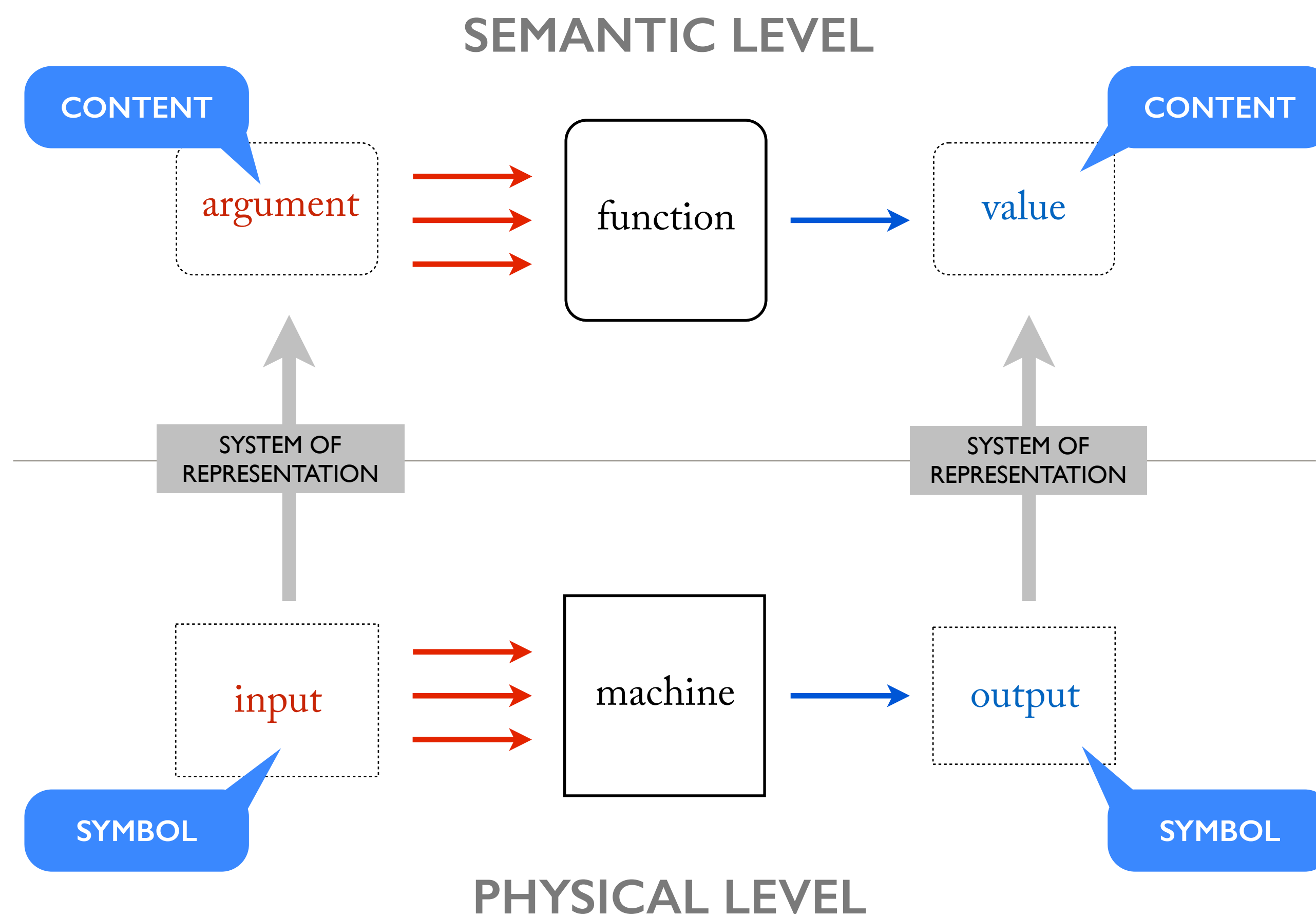


5. Systems of Representation

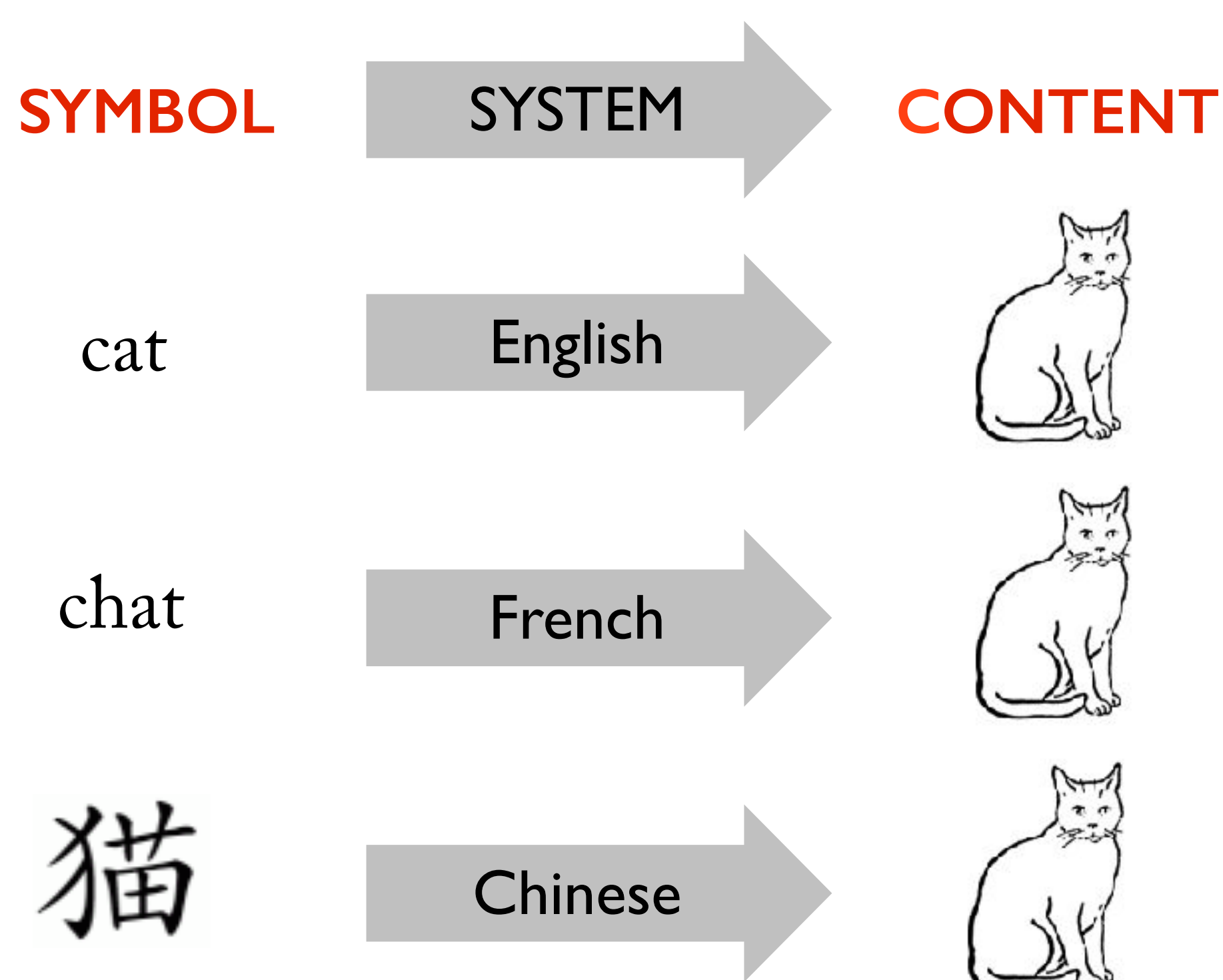
The role of representational systems in computation

Systems of representation associate symbols with contents. In computation, they map inputs to arguments and outputs to values. They are the glue that connects the physical and semantic levels.



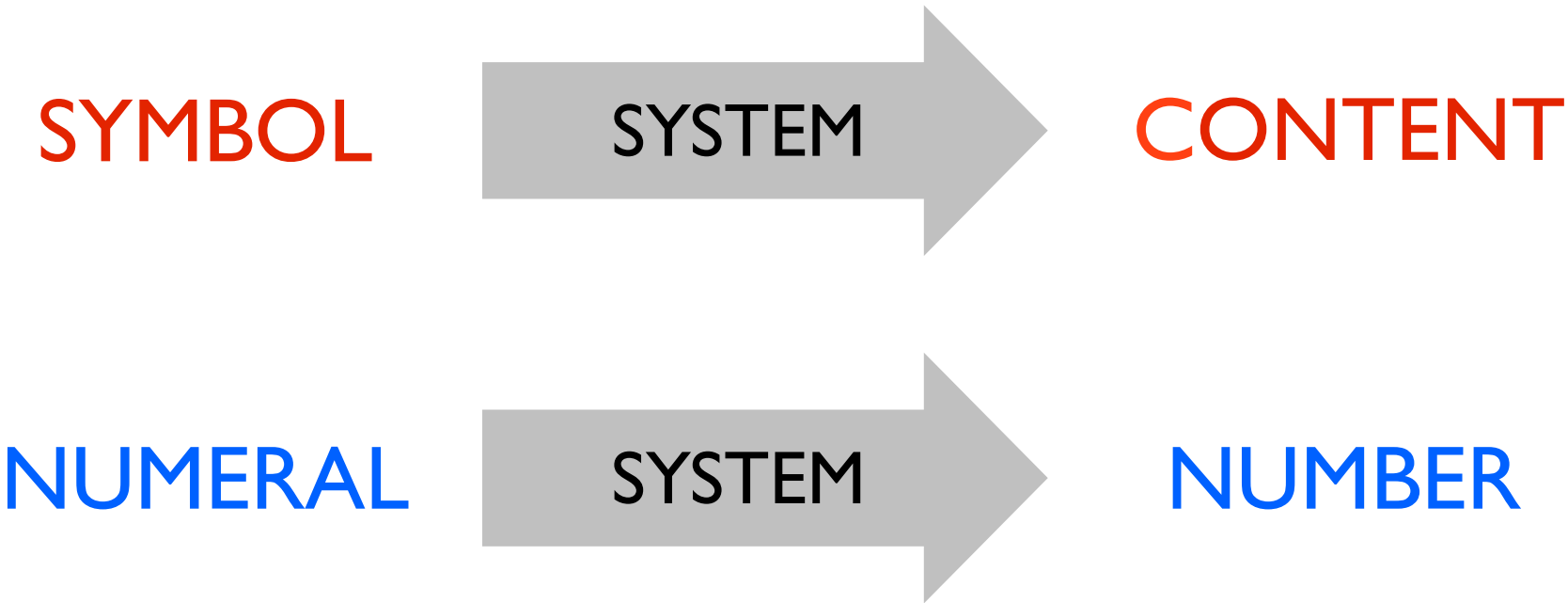
Symbol, content, system

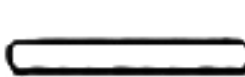
Symbols express contents only relative to a system. The same content can be expressed by different symbols in different systems. Here is an illustration for the case of words in a language.

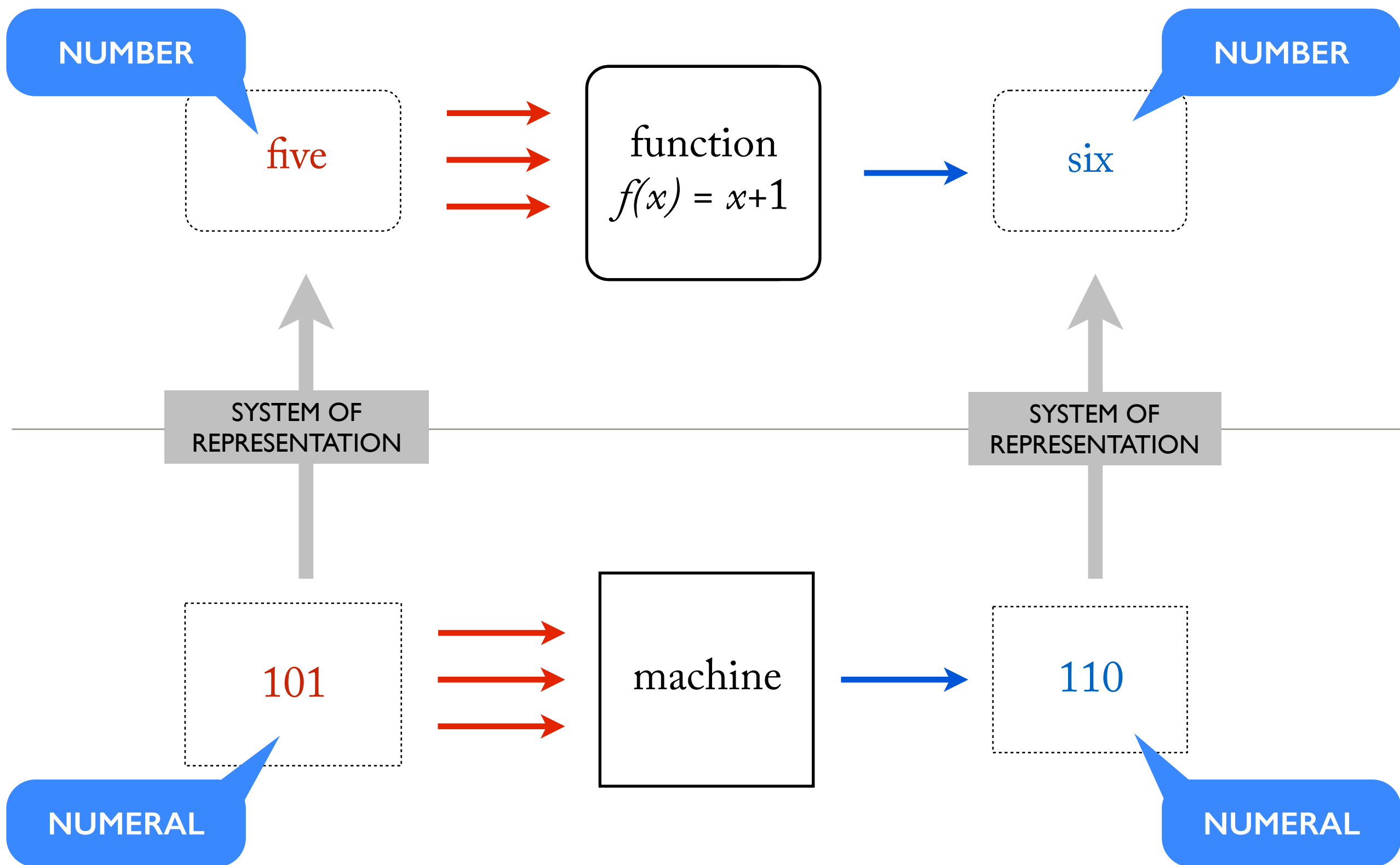


Numeral systems

Numeral systems are systems of representation specifically for numbers. In these systems, the symbols are called **numerals**, and the contents are **numbers**. Numerals are types of physical symbols, while numbers are abstract quantities.

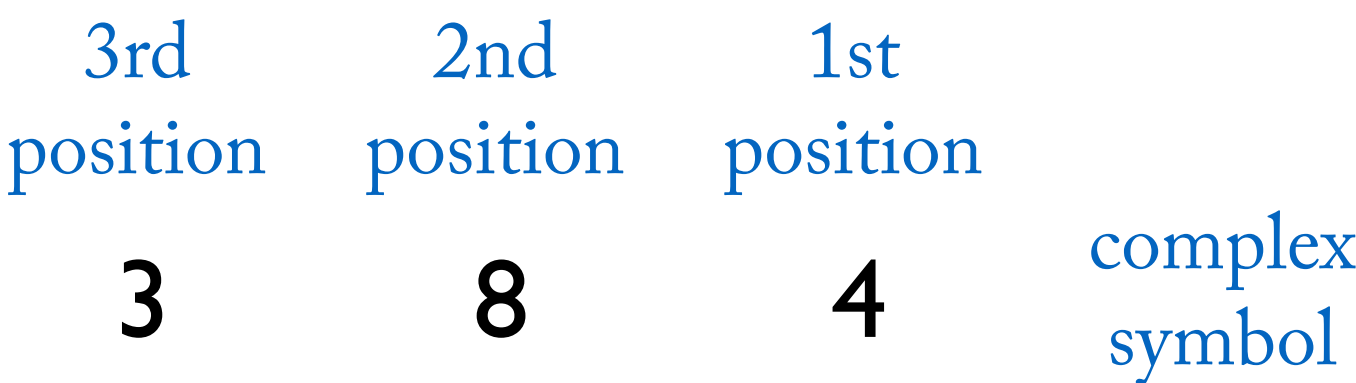


Different numeral systems use....	Roman numerals	Arabic numerals	Mayan numerals	The numbers represented
		0		<i>zero</i>
	I	1		<i>one</i>
	II	2		<i>two</i>
	III	3		<i>three</i>
	IV	4		<i>four</i>
	V	5		<i>five</i>



Anatomy of a Numeral

A **complex** numeral is a made up of a string of **atomic** numerals. Each atomic numeral has a **position** within the complex numeral.



In the Arabic numeral system, the atomic numerals are the numerals 0-9, and all other numerals are constructed from these.

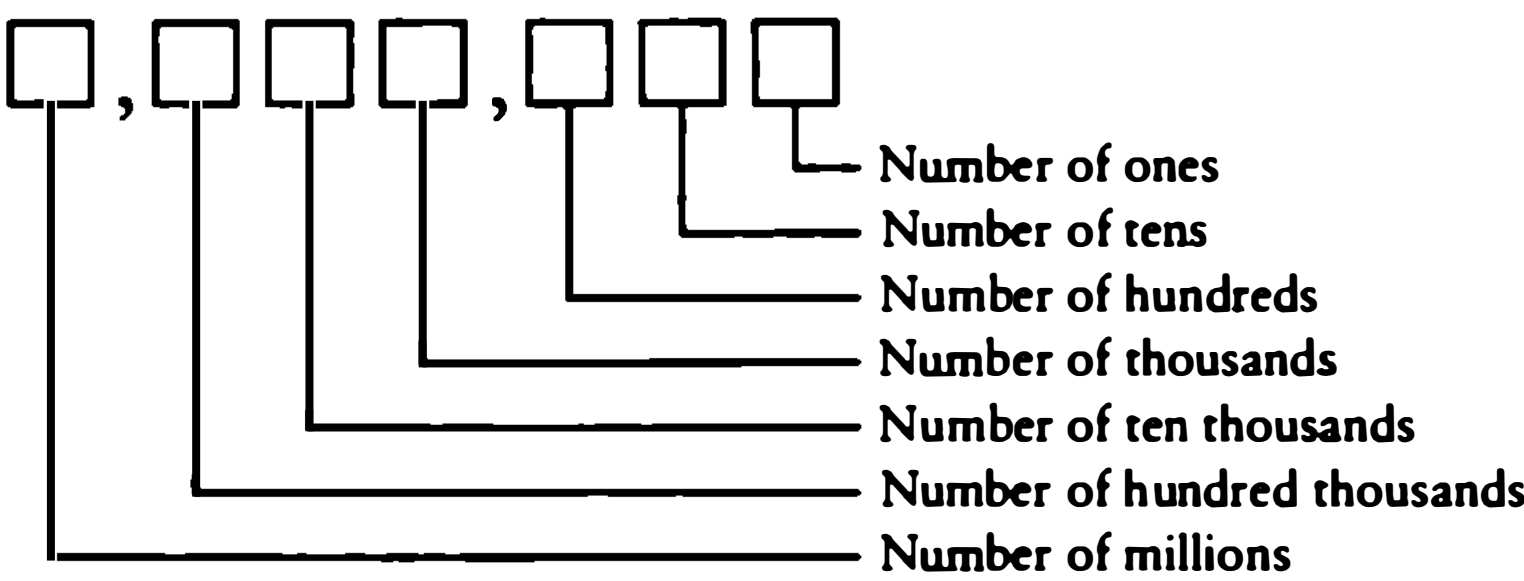
Atomic symbols in the **Indo-Arabic** system

0 1 2 3 4 5 6 7 8 9

Mechanics of a Numeral System

A **positional** numeral system is defined by (i) the values it assigns to each atomic symbol (always starting with zero); (ii) the value of its positions.

Positions in the **Indo-Arabic** system



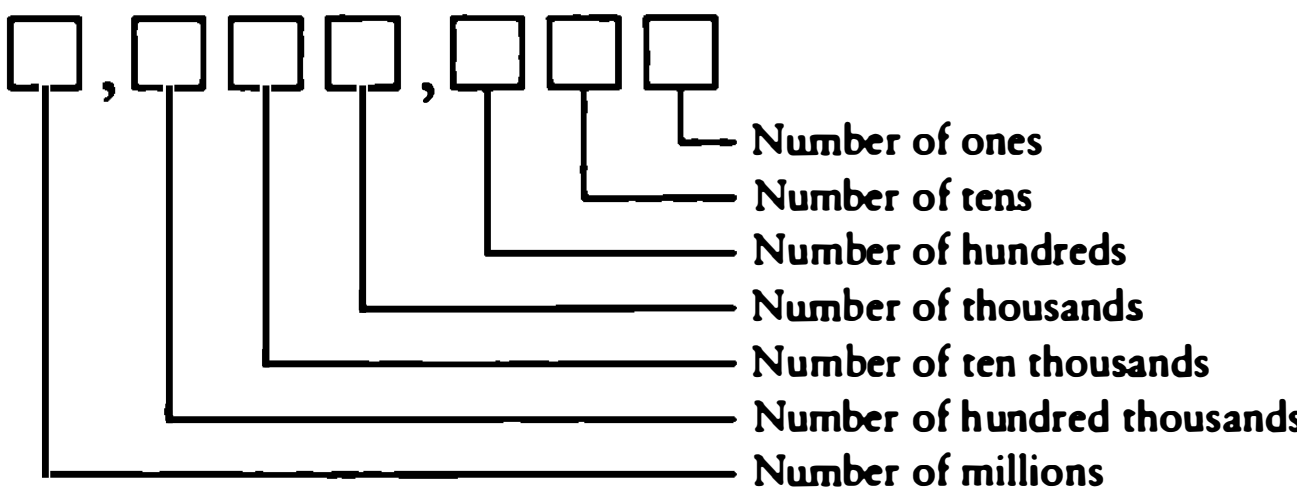
The Arabic numeral system is positional, but the Roman system is not.

Base

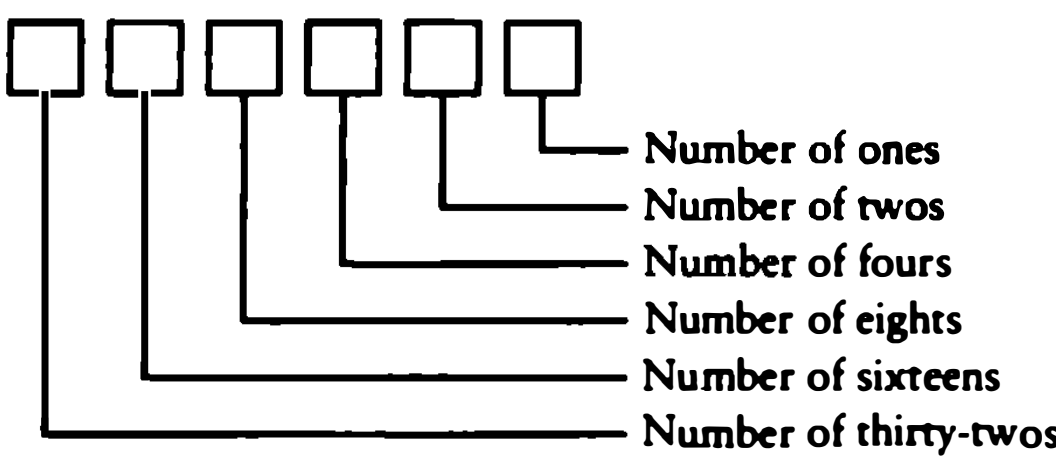
The **base** of a positional system corresponds to the number of atomic symbols. In the Arabic system, there are ten symbols, 0-9, so this system is called **base 10**. In the Binary system, there are only two symbols (0,1), so this system is **base 2**.

The base determines the value of each position.

Base 10



Base 2 (Binary)



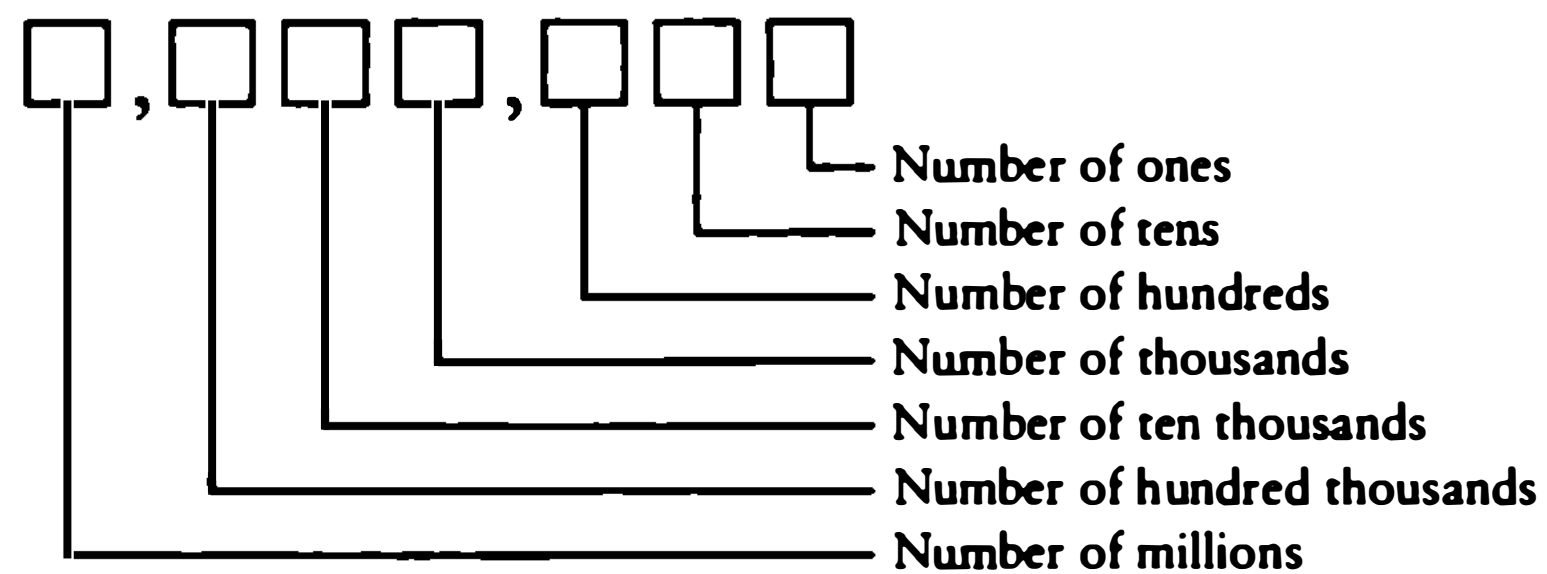
Base 10	Base 7	Base 2 (Binary)	Base 1 (Tally)
1	1	1	1
2	2	10	11
3	3	11	111
4	4	100	1111
5	5	101	11111

Calculating a Numeral's Value

Decimal System

ten atomic symbols, $base=10$

0 1 2 3 4 5 ...
zero one two three four five ...

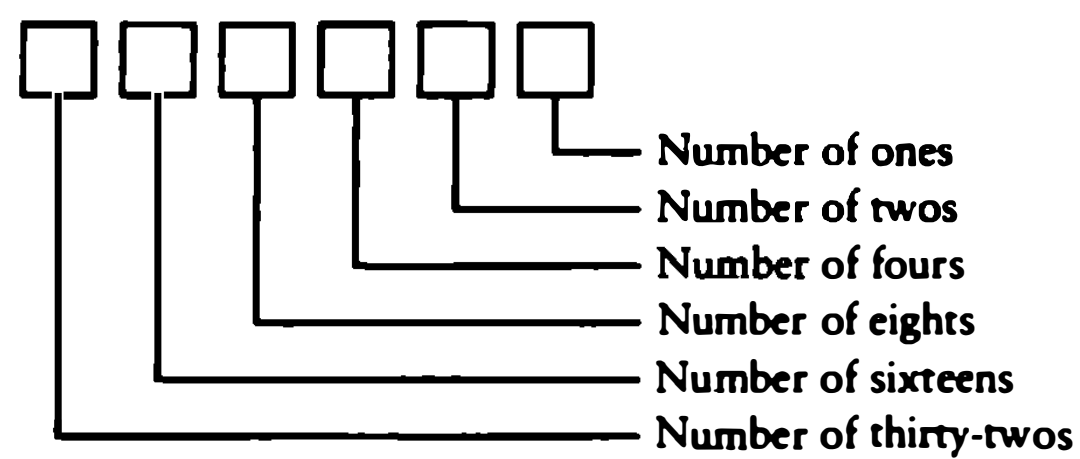


$$111_D = 1_D \times 100 + 1_D \times 10 + 1_D \times 1$$

Binary System

two atomic symbols, $base = 2$

0 1
zero one

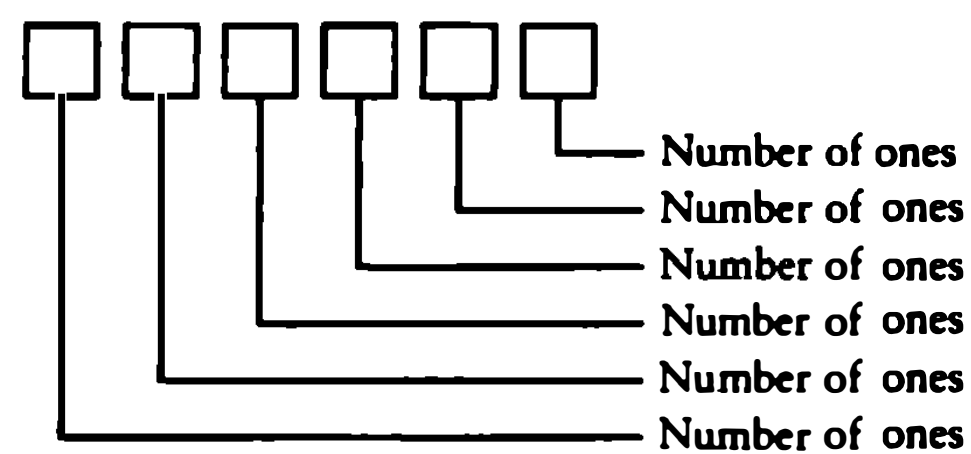


$$111_B = 1_B \times 4 + 1_B \times 2 + 1_B \times 1$$

Tally System

one atomic symbol, $base = 1$

|



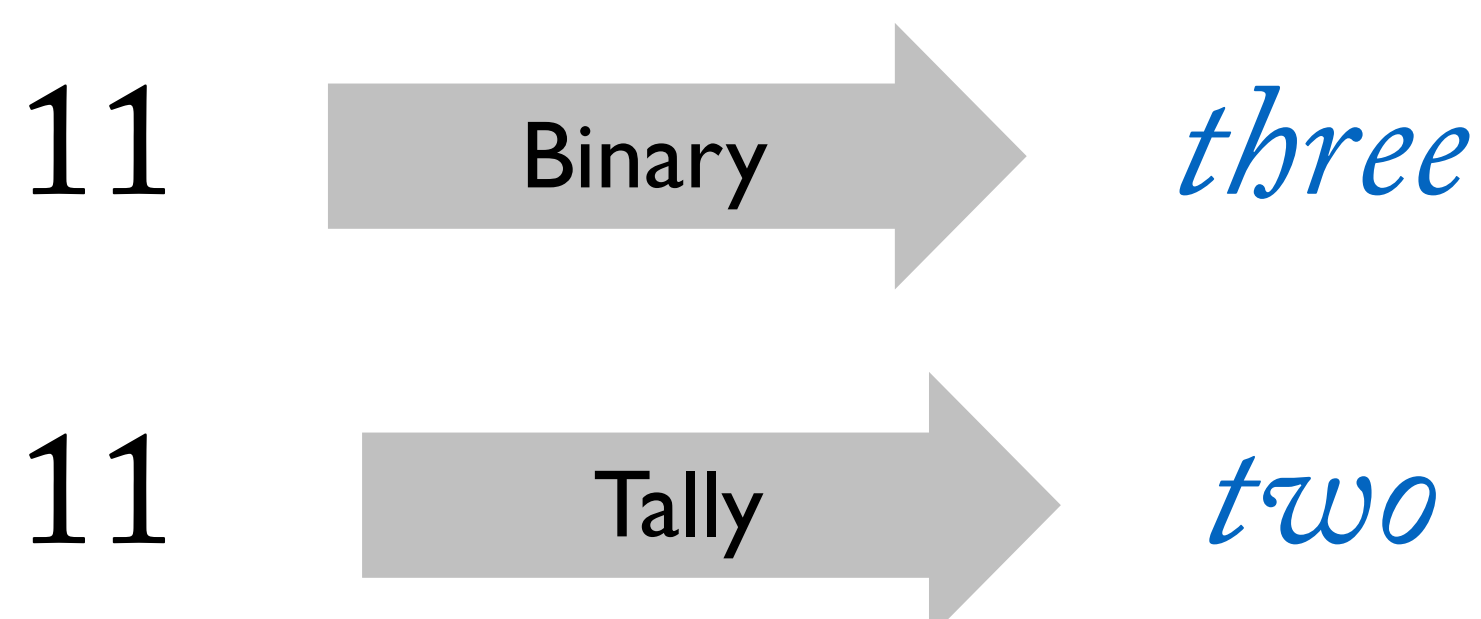
$$111_T = 1_T \times 1 + 1_T \times 1 + 1_T \times 1$$

Note: extended tally system with unlimited 0's to the left.

Binary and Tally

In this course, the most important numeral systems will be **tally (T)** and **binary (B)**. This is because you can express all numbers in these systems using only two symbols, 0 and 1.

The same set of numerals represent different numbers in the two systems.



Tally Syntax

Tally systems in common usage take a variety of forms.

For our purposes a **well formed tally numeral** is any string of bits, where the first n (≥ 0) bits from the right are 1's, and they are followed to the left by any number of 0's.

(Images
from
Wikipedia)

0 . . . 0 1 . . . 1

So the following
are well formed:

000

11

0011

1111

But these ones
are not.

1000

1101

1110

When designing machine to compute a function in tally, we'll always assume that inputs are well-formed.

Binary Syntax

Tally systems in common usage take a variety of forms.

For our purposes a **well formed tally numeral** is any string of bits, where the first n (≥ 0) bits from the right are 1's, and they are followed to the left by any number of 0's.

0 . . . 0 1 . . . 1

So the following are well formed:

000	
11	000
0011	11
1111	0011
	1111

When designing machine to compute a function in tally, we'll assume that all inputs are well-formed. For example, you'll never get the input "1011".

(Images
from
Wikipedia)

one